

Assignment 2

```
In [38]: import numpy as np

# Given 4x4 matrix A
A = np.array([
    [4, -2, 2, 1],
    [1, 1, 0, 1],
    [-2, 1, 3, -1],
    [1, 3, -1, 2]
]);
```

1) Calculating the inverse matrix of A (1 pts)

```
In [39]: A_inv = np.linalg.inv(A)

print("Inverse of A:")
print(A_inv)
# print(
#     '\n',
#     "A @ A_inv is identical to A's identity matrix?",
#     np.allclose(A @ A_inv, np.eye(4))
# )
```

Inverse of A:

```
[[ 5.00000000e+00 -3.00000000e+01  1.00000000e+00  1.30000000e+01]
 [ 4.00000000e+00 -2.50000000e+01  1.00000000e+00  1.10000000e+01]
 [-1.00000000e+00  7.00000000e+00  5.82867088e-16 -3.00000000e+00]
 [-9.00000000e+00  5.60000000e+01 -2.00000000e+00 -2.40000000e+01]]
```

2) Calculating Eigenvalues and Eigenvectors for the 4x4 Matrix (2 pts).

```
In [40]: λ, v = np.linalg.eig(A)

print('eigenvalues:')
print(λ)
print('\neigenvectors:')
print(v)
# print('\nAv is identical to λv?', np.allclose(A @ v, λ * v))
```

```
eigenvalues:
[[-0.02231724+0.j          3.6104532 +1.72034086j   3.6104532 -1.72034086
j
 2.80141085+0.j          ]

eigenvectors:
[[-0.43676159+0.j          0.57598271+0.j          0.57598271-0.j
 0.26748859+0.j          ]
 [-0.36900111+0.j          0.00962337-0.2105225j   0.00962337+0.2105225
j
 -0.34893941+0.j          ]
 [ 0.10239683+0.j          -0.00821729+0.55142262j  -0.00821729-0.5514226
2j
 -0.06120796+0.j          ]
 [ 0.81399778+0.j          -0.18869091-0.53300366j  -0.18869091+0.5330036
6j
 -0.89607183+0.j          ]]
```

3) After having the Eigenvalues and Eigenvectors,

- Constructing the eigen matrix P from eigenvectors: `P = eigenvectors`
- Constructing the diagonal matrix D from eigenvalues: `D = np.diag(eigenvalues)`
- Calculating the inverse of the eigen matrix: `P_inv = np.linalg.inv(P)`

Verify if the diagonalized matrix `A_diagonalized` is identical to the original matrix `A` (2 pts).

```
In [41]: P = v
D = np.diag(λ)
P_inv = np.linalg.inv(P)
A_diagonalized = P @ D @ P_inv

print(A_diagonalized);
print('\nA is identical to A_diagonalized?', np.allclose(A, A_diagonalized))

[[ 4.00000000e+00-8.99194310e-17j -2.00000000e+00+1.73109402e-16j
 2.00000000e+00+1.61726207e-16j  1.00000000e+00-1.53633405e-16j]
 [ 1.00000000e+00-1.04041487e-16j  1.00000000e+00-4.65692989e-18j
 -4.13857983e-15-5.94294828e-17j  1.00000000e+00+4.49098464e-17j]
 [-2.00000000e+00-5.33113837e-16j  1.00000000e+00+1.69040751e-16j
 3.00000000e+00+3.11083746e-16j -1.00000000e+00-1.57281856e-16j]
 [ 1.00000000e+00-6.12647601e-16j  3.00000000e+00-1.44359334e-16j
 -1.00000000e+00-2.91727013e-16j  2.00000000e+00+1.21740238e-16j]]
```

A is identical to A_diagonalized? True

4) Explain why this result (a diagonalized matrix is identical to the original matrix) is important in linear algebra (1 pts).

This is important because it proves that the original matrix can be broken down into its fundamental components, the eigenvalues and eigenvectors. When a matrix is diagonalized, performing operations such as Principal Component Analysis is much easier, we simply use the diagonalized components instead of performing expensive matrix operations. Being able to reconstruct the original matrix from its diagonalized components proves that we have accounted for all the variance in the original data when we perform dimensionality reduction.

5) Make a python code using SVD example to get the same A (AR) (4 pts)

- Decide your own range for the sigma

```
In [42]: from scipy.linalg import svd as SVD

U, S, V = SVD(A)
sigma = np.zeros((4, 4))
# The matrix is very small. We use the full range of the sigma, which
sigma_range = 4

for i in range(sigma_range):
    sigma[i, i] = S[i]

AR = U @ sigma @ V

print('A:')
print(A)
print('\nAR:')
print(AR)
print('\nA is identical to AR?', np.allclose(A, AR))
```

A:

```
[[ 4 -2  2  1]
 [ 1  1  0  1]
 [-2  1  3 -1]
 [ 1  3 -1  2]]
```

AR:

```
[[ 4.00000000e+00 -2.00000000e+00  2.00000000e+00  1.00000000e+00]
 [ 1.00000000e+00  1.00000000e+00 -3.3770876e-16  1.00000000e+00]
 [-2.00000000e+00  1.00000000e+00  3.00000000e+00 -1.00000000e+00]
 [ 1.00000000e+00  3.00000000e+00 -1.00000000e+00  2.00000000e+00]]
```

A is identical to AR? True